

IQR · INTELLIGENT QUALITY REVIEW

Migration & Deployment Guide

From an empty machine to a fully working, Azure-connected agentic validation platform — every step as a terminal command, with what it does and what you should see. No portal clicking required.

Repository https://github.com/git-bonda108/IQR_Agentic_AI

Version 1.0 · September 2026 · Applies to macOS, Linux, and Windows

Companion documents: Design Document v4 · UAT Guide · Production Standards

Contents

1. What you are building — the modules and what each accomplishes
2. Prerequisites — install the five tools
3. Clone and install
4. Prove it offline first (no cloud, no keys)
5. Sign in to Azure — and what to do when the login or tenant changes
6. Create every Azure resource from the command line
7. Model deployments — which model in which seat, and why
8. Connect the machine: the .env file, variable by variable
9. Upload the data estate — what goes in which container
10. Seed the knowledge index (Foundry IQ)
11. Run the platform end to end
12. Deploy the web console to Azure + secure it (app registration explained)
13. Identity & tenant-change playbook
14. Troubleshooting
15. Appendix — the whole thing as one command sequence

1 · What you are building

IQR validates internal-control evidence packages end to end: a reviewer uploads the period's evidence (workbooks, approval emails, ZIPs, screenshots, in any mix), IQR understands the package, runs the frozen validation plan, and produces an audit-ready pack in which every claim carries a citation to the exact cell, email line, or image it came from. Each module below is something you will touch during this guide:

MODULE	WHAT IT ACCOMPLISHES
<code>iqr/ingest + iqr/intake.py</code>	Unpacks the nested evidence tree, SHA-256 fingerprints every file, extracts every cell/email line/image, infers which control the package belongs to, and narrates what it found — before anything runs
<code>iqr/graph</code>	The fixed validation pipeline: ingest → match → sentinel → checks (parallel) → blinded verify → adjudicate. Match works by filename first, then by meaning (embeddings)
<code>iqr/tools + iqr/checks</code>	Deterministic tools compute every number/timestamp; AI agents decide only what evidence means. Numbers are never model output
<code>iqr/agents</code>	The model client: routes each agent seat to its Azure AI Foundry deployment, with a deterministic offline stub as permanent fallback
<code>iqr/knowledge</code>	Control knowledge base + Golden Library (adjudication precedents) + the embedding seat, served by Azure AI Search
<code>iqr/eval + iqr/learn</code>	The five-gate evaluation harness (with batch scoring and per-check confidence) and the reinforcement learner that turns human adjudications into earned confidence
<code>iqr/api + webapp/</code>	The reviewer console: upload → package story → run → live ledger → verdict → audit pack → governance
<code>scripts/azure</code>	Provisioning, knowledge seeding, and web-app bundling scripts used throughout this guide

2 · Prerequisites — install the five tools

Why: Python runs the platform; git fetches it; the Azure CLI creates and connects cloud resources from the terminal; Tesseract performs OCR for screenshot checks; VS Code is optional but convenient.

macOS (using Homebrew — install it first from `brew.sh` if absent):

```
brew install python@3.12 git azure-cli tesseract
```

Windows (PowerShell, using the built-in winget):

```
winget install Python.Python.3.12 Git.Git Microsoft.AzureCLI UB-Mannheim.TesseractOCR
```

Linux (Debian/Ubuntu; Azure CLI via Microsoft's script):

```
sudo apt update && sudo apt install -y python3.12 python3.12-venv git tesseract-ocr
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Verify each tool answers (run all four):

```
python3 --version && git --version && az version && tesseract --version
```

Expected: four version strings, no “command not found”. Python must be 3.12 or newer.

Do not clone into a cloud-synced folder (OneDrive, iCloud Drive). File-sync tools evict file contents to the cloud and reads stall for minutes. Use a plain path such as `C:\src` or `~/src`.

Windows note: wherever this guide writes `.venv/bin/python`, type `.venv\Scripts\python`.

3 · Clone and install

Why: the repository is the entire deployable unit — code, tests, frozen plans, documentation, provisioning scripts. The virtual environment (`.venv`) is a private Python installation for this project so nothing touches your system Python.

```
git clone https://github.com/git-bondal08/IQR_Agentic_AI.git IQR
cd IQR
python3 -m venv .venv
.venv/bin/pip install -e .
```

Expected: the clone prints object counts; pip resolves ~25 packages and ends with “Successfully installed iqr-0.1.0 ...”. Takes 2–5 minutes on a normal connection.

4 · Prove it offline first — no cloud, no keys

Why: IQR ships a deterministic offline model stub, so the entire platform — ingestion, checks, verification, evaluation — runs with zero network access. If this step is green, the platform is healthy on this machine, and any later problem is by definition an Azure-connection problem, not a code problem. This is your acceptance gate.

```
.venv/bin/python tests/fixtures/build_fixtures.py
.venv/bin/python -m pytest tests/ -q
```

Expected: the first command builds three synthetic “golden” control packages silently; the second prints `61 passed` in well under a minute.

Optionally see the console right now, fully offline:

```
.venv/bin/python -m iqr.cli serve
```

Expected: “Uvicorn running on http://127.0.0.1:8400” — open that address in a browser, drag the contents of `tests/fixtures/controls/C23024/package` into the upload zone, and watch intake, validation, and the audit pack work with no cloud at all. Ctrl-C stops the server.

5 · Sign in to Azure — and when the login or tenant changes

Why: everything cloud-side happens under an Azure *subscription* (the billing boundary) inside a *tenant* (the identity boundary — one organization's directory of users and apps). The CLI must hold a token for the right tenant and subscription before any resource command works.

```
az login
```

Expected: a browser window opens for sign-in (with MFA if the tenant requires it); the terminal then lists the subscriptions your account can use and marks the default with `*`.

Pin the subscription explicitly (never rely on the default):

```
az account set --subscription "<subscription name or id>"
az account show --query "{user:user.name, tenant:tenantId, sub:name}" -o json
```

Expected: JSON echoing your signed-in user, tenant id, and subscription — confirm all three before continuing.

If the Azure login or tenant changes later

Nothing in the code is tenant-specific. Only three things are bound to a tenant, and all three are rebuilt by commands in this guide:

TENANT-BOUND ITEM	WHAT TO DO IN THE NEW TENANT
The CLI token	<code>az logout</code> then <code>az login --tenant <new-tenant-id></code>
Role assignments (who may write blobs, etc.)	Re-run the <code>az role assignment create</code> commands in §9 for the new identities
The console's app registration (§12)	Create a new registration in the new tenant and update the web app's auth settings — two commands, shown in §12

If resources themselves move to a new subscription, simply re-run §6–§10 there — provisioning is scripted and idempotent, and the data estate re-uploads from your local `data/` tree or transfers with `azcopy`. Budget one hour for a complete rebuild in a fresh subscription.

6 · Create every Azure resource from the command line

Why each resource exists: a **resource group** is a folder holding everything (delete it, and everything goes — clean teardown); a **storage account** holds blob containers (files) and tables (rows); an **AI Foundry account + project** hosts the model deployments; **AI Search** hosts the knowledge index the agents retrieve against.

6.1 Set your names once (storage and search names are global across Azure — add your own suffix; lowercase, no dashes in the storage name):

```
export LOC=eastus2 RG=rg-iqr-sox SA=stiqrsox<suffix> FOUNDRY=iqr-foundry-<suffix> SEARCH=iqr-search-<suffix>
```

6.2 Resource group:

```
az group create -n $RG -l $LOC
```

Expected: JSON with `"provisioningState": "Succeeded"`.

6.3 Storage account, blob containers, and table storage. The six containers map to IQR's stores; the two tables hold the run index and the human-in-the-loop exception queue:

```
az storage account create -n $SA -g $RG -l $LOC --sku Standard_LRS \
  --kind StorageV2 --min-tls-version TLS1_2 --allow-blob-public-access false

KEY=$(az storage account keys list -n $SA -g $RG --query "[0].value" -o tsv)

for c in evidence-store run-ledgers audit-packs plans knowledge source-evidence; do
  az storage container create -n $c --account-name $SA --account-key "$KEY"
done

az storage table create -n runs --account-name $SA --account-key "$KEY"
az storage table create -n exceptions --account-name $SA --account-key "$KEY"
```

Expected: `"created": true` once per container and table (eight in total).

STORE	HOLDS
evidence-store	Immutable evidence files named by their SHA-256 — the chain of custody
run-ledgers	Append-only JSONL of every run — the replayable audit trail
audit-packs / plans	Deliverable zips · frozen approved validation plans
knowledge / source-evidence	Knowledge documents · raw incoming packages
tables: runs / exceptions	Queryable run index · exception queue rows (pennies to operate)

6.4 AI Foundry account and project:

```
az cognitiveservices account create -n $FOUNDRY -g $RG -l $LOC \
  --kind AIServices --sku S0 --custom-domain $FOUNDRY

SUB=$(az account show --query id -o tsv)
az rest --method put \
  --url "https://management.azure.com/subscriptions/$SUB/resourceGroups/$RG/providers/Microsoft.CognitiveServices/accounts/$FOUNDRY/projects/iqr-sox?api-version=2025-04-01-preview" \
  --body '{"location":"'$LOC'", "identity":{"type":"SystemAssigned"}, "properties":{"description":"IQR validation"}}'
```

Expected: both return `"provisioningState": "Succeeded"`. If the account name is taken, change the suffix.

6.5 AI Search (free tier is enough to start; if the region is out of capacity, retry with another region for this one service):

```
az search service create -n $SEARCH -g $RG -l $LOC --sku free
```

Expected: "provisioningState": "succeeded" (this one can take a few minutes).

Shortcut: `bash scripts/azure/provision.sh` runs 6.2–6.5 in one shot and prints the .env block to paste. The manual steps above are what it does inside.

7 · Model deployments — which model in which seat, and why

Why multiple models: IQR routes each agent “seat” to the model best suited to it. Judgment-light check agents need speed, low cost, and — critically — **temperature 0** for reproducibility. Verification benefits from a deeper reasoning model. Evidence matching needs an embedding model, not a chat model.

```
az cognitiveservices account deployment create -n $FOUNDRY -g $RG \
--deployment-name gpt-41-mini --model-name gpt-4.1-mini --model-version 2025-04-14 \
--model-format OpenAI --sku-name GlobalStandard --sku-capacity 10

az cognitiveservices account deployment create -n $FOUNDRY -g $RG \
--deployment-name gpt-5-mini --model-name gpt-5-mini --model-version 2025-08-07 \
--model-format OpenAI --sku-name GlobalStandard --sku-capacity 50

az cognitiveservices account deployment create -n $FOUNDRY -g $RG \
--deployment-name model-router --model-name model-router --model-version 2025-11-18 \
--model-format OpenAI --sku-name GlobalStandard --sku-capacity 10

az cognitiveservices account deployment create -n $FOUNDRY -g $RG \
--deployment-name text-embedding-3-small --model-name text-embedding-3-small \
--model-version 1 --model-format OpenAI --sku-name GlobalStandard --sku-capacity 100
```

Expected: Succeeded four times. If you see `InsufficientQuota`, list what your subscription can deploy with `az cognitiveservices usage list -l $LOC -o table` and substitute a model with headroom; if you see `ServiceModelDeprecating`, use the newer version the error implies.

DEPLOYMENT	SEAT	WHY THIS MODEL
gpt-41-mini	Vision, temporal, sign-off check agents	Fast, inexpensive, supports temperature 0 + seed — the reproducibility law binds these seats
gpt-5-mini	Blinded verifier	Reasoning-class skeptic; re-performs findings from citations alone. (Reasoning models don't accept temperature 0 — acceptable here because the verifier only re-derives from deterministic tool output)
model-router	Plan compiler (design time)	Routes each request to the best underlying model for quality — worth it for the hardest, least-frequent task
text-embedding-3-small	Semantic evidence matching	Relates real-world filenames and content (sheet names, email subjects) to what the plan expects; cheap at scale

Model governance: when a newly approved model becomes available, it earns a seat through a fixed graduation: add its deployment → route one seat via `IQR_FOUNDRY_DEPLOYMENT_<SEAT>` → run `.venv/bin/python -m iqr.cli eval --batch 3` → all five gates green → keep it. The run ledger attributes every call to its backend, so a seat change is an auditable event.

8 · Connect the machine: the .env file, variable by variable

Why: `.env` holds this machine's connection settings. It is listed in `.gitignore` and must never be committed — the repository carries a template instead.

```
cp .env.example .env
```

Fetch the three secrets from the CLI and edit `.env`:

```
az cognitiveservices account keys list -n $FOUNDRY -g $RG --query key1 -o tsv
az search admin-key show --service-name $SEARCH -g $RG --query primaryKey -o tsv
az storage account keys list -n $SA -g $RG --query "[0].value" -o tsv
```

VARIABLE	SET TO / MEANING
IQR_MODEL	auto — try Foundry first, fall back to the offline stub (never blocks)
AZURE_FOUNDRY_ENDPOINT	https://<foundry-name>.openai.azure.com
AZURE_FOUNDRY_API_KEY	first key from the first command above
AZURE_FOUNDRY_DEPLOYMENT	gpt-41-mini (the default seat)
AZURE_FOUNDRY_EMBEDDING_DEPLOYMENT	text-embedding-3-small
IQR_FOUNDRY_DEPLOYMENT_VERIFY	gpt-5-mini — per-seat routing for the verifier
IQR_FOUNDRY_DEPLOYMENT_PLAN_COMPILE	model-router
FOUNDRY_IQ_ENDPOINT / _API_KEY	https://<search-name>.search.windows.net / the search admin key
FOUNDRY_IQ_KNOWLEDGE_BASE	iqr-knowledge
IQR_AZ_STORAGE_ACCOUNT / _KEY	the storage name / key

Prove the connection:

```
.venv/bin/python -m iqr.cli testmodel
```

Expected: answered by: foundry . If it says answered by: stub , an endpoint or key in `.env` is wrong — the platform still works, it is telling you visibly that it fell back.

9 · Upload the data estate — what goes in which container

Why sign-in-based upload: granting your own account a data role and using `--auth-mode login` keeps storage keys off the command line entirely.

```
SUB=$(az account show --query id -o tsv)
ME=$(az ad signed-in-user show --query id -o tsv)
az role assignment create --assignee $ME --role "Storage Blob Data Contributor" \
  --scope /subscriptions/$SUB/resourceGroups/$RG/providers/Microsoft.Storage/storageAccounts/$SA
```

Expected: JSON describing the assignment. Allow a minute to propagate before the first upload.

```
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s data/plans -d plans --overwrite --no-progress
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s data/runs -d run-ledgers --overwrite --no-progress
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s data/packs -d audit-packs --overwrite --no-progress
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s tests/fixtures/controls -d source-evidence --destination-path fixtures --overwrite --no-progress
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s data/input -d source-evidence --destination-path input --overwrite --no-progress
az storage blob upload-batch --auth-mode login --account-name $SA \
  -s data/evidence_store -d evidence-store --overwrite --no-progress
```

Expected: each command prints one JSON entry per uploaded file. Verify a count: `az storage blob list --auth-mode login --account-name $SA -c plans --query "length(@)"`.

Note: the `data/` tree is gitignored — real evidence never travels through git. A brand-new clone gets its data by downloading from these containers (`az storage blob download-batch` , same flags with `-s` and `-d` swapped) or starts fresh with the synthetic fixtures, which need nothing.

10 · Seed the knowledge index (Foundry IQ)

Why: the plan-compiler and check/verify agents ground themselves by retrieving from a knowledge index — approved plans, real-corpus lessons, adjudication precedents — instead of free-associating. This script creates the index and loads it; rerun it any time plans change (idempotent).

```
./venv/bin/python scripts/azure/seed_knowledge.py
```

Expected: index 'iqr-knowledge': 12/12 documents indexed (4 plans, 8 lessons, 0 exemplars) — numbers grow as plans and released exemplars accumulate.

11 · Run the platform end to end

```
.venv/bin/python -m iqr.cli serve
```

Open `http://127.0.0.1:8400`, drag the contents of `tests/fixtures/controls/C10032/package` into the upload zone, and follow the flow: intake analyzes the package, suggests the control with an evidence-coverage score, tells you the package's story, and — after you confirm — runs the frozen plan with the ledger streaming live.

Expected: verdict **pass**, findings with citations like `Recon!B30339`, a downloadable audit pack, and agent lines reading `agent[foundry]` — proof the cloud models are doing the reasoning.

Then run the quality gates (the release bar for any future change):

```
.venv/bin/python -m iqr.cli eval  
.venv/bin/python -m iqr.cli eval --batch 3
```

Expected: `ALL GATES PASS`; the batch adds per-gate mean/min/max and a per-check confidence table (HIGH means the verdict was identical across every repeat). The full 13-case acceptance script is `docs/UAT.md`.

12 · Deploy the web console to Azure and secure it

12.1 Create the hosting plan (free tier) and the web app, then tell it how to start:

```
az appservice plan create -n plan-iqr -g $RG -l $LOC --is-linux --sku F1
az webapp create -n <app-name> -g $RG --plan plan-iqr --runtime "PYTHON:3.12"
az webapp config set -n <app-name> -g $RG \
  --startup-file "python -m uvicorn iqr.api.app:app --host 0.0.0.0 --port 8000"
```

12.2 Give the app its settings (same values as .env — App Service settings are the cloud's version of the .env file; secrets never go into the code bundle):

```
az webapp config appsettings set -n <app-name> -g $RG --settings \
  IQR_MODEL=auto AZURE_FOUNDRY_ENDPOINT=... AZURE_FOUNDRY_API_KEY=... \
  AZURE_FOUNDRY_DEPLOYMENT=gpt-41-mini AZURE_FOUNDRY_EMBEDDING_DEPLOYMENT=text-embedding-3-small \
  IQR_FOUNDRY_DEPLOYMENT_VERIFY=gpt-5-mini FOUNDRY_IQ_ENDPOINT=... FOUNDRY_IQ_API_KEY=... \
  FOUNDRY_IQ_KNOWLEDGE_BASE=iqr-knowledge SCM_DO_BUILD_DURING_DEPLOYMENT=true
```

12.3 Build the deployment bundle (code + frozen plans + fixtures, no secrets) and ship it:

```
bash scripts/azure/build_webapp_bundle.sh
az webapp deploy -n <app-name> -g $RG --src-path deploy.zip --type zip
```

Expected: “deploy.zip ready”, then a deployment that finishes cleanly; a minute later `curl https://<app-name>.azurewebsites.net/api/topology` returns JSON.

12.4 · Lock it down — the app registration, explained

What an app registration is and why you need one: it is the tenant's record that “this web app is allowed to ask our directory to sign users in.” The registration owns a client id and the redirect URL the sign-in flow returns to. App Service authentication (“Easy Auth”) uses it to force a directory sign-in in front of every request — no anonymous access, and no auth code inside IQR itself. In a centrally managed tenant, creating registrations may be restricted: request one from the directory admins with the redirect URL below, or ask them to grant your account the ability to create it.

```
APPID=$(az ad app create --display-name iqr-console \
  --web-redirect-uris https://<app-name>.azurewebsites.net/.auth/login/aad/callback \
  --enable-id-token-issuance true --sign-in-audience AzureADMyOrg --query appId -o tsv)
TENANT=$(az account show --query tenantId -o tsv)

az webapp auth config-version upgrade -n <app-name> -g $RG
az webapp auth microsoft update -n <app-name> -g $RG \
  --client-id $APPID --issuer https://login.microsoftonline.com/$TENANT/v2.0 --yes
az webapp auth update -n <app-name> -g $RG --enabled true \
  --action RedirectToLoginPage --redirect-provider AzureActiveDirectory
```

Expected: after ~a minute, `curl -s -o /dev/null -w "%{http_code}" https://<app-name>.azurewebsites.net/` prints **401** (anonymous rejected), while a browser gets the organization's sign-in page and signed-in members reach the console.

13 · Identity & tenant-change playbook

SCENARIO	WHAT TO DO
Same tenant, new person joins	They run §2–§4, then §8 with the shared resource names. Grant them the storage data role (§9, with their <code>az ad signed-in-user show id</code>) if they will upload evidence
New subscription, same tenant	<code>az account set --subscription <new></code> , re-run §6–§10 (an hour, all scripted). App registration carries over unchanged
New tenant entirely (e.g., moving from a personal to an organizational directory)	<code>az logout</code> ; <code>az login --tenant <new-id></code> ; re-run §6–§12. Create the app registration in the new tenant (§12.4) — or request it from the directory admins if self-service registration is disabled — and update the web app's issuer/client id with the same two commands
Organization requires admin approval for apps	Send the admins: the app name, the redirect URL (<code>https://<app-name>.azurewebsites.net/.auth/login/aad/callback</code>), “ID tokens enabled, single-tenant.” That is everything they need to create/approve it
Keys are prohibited by policy	Switch service-to-service auth to managed identities: <code>az webapp identity assign</code> , grant that identity the storage/search/Foundry data roles, and remove the key variables — the production-standards document (§4) lists the exact roles
MFA / Conditional Access enforced	Nothing changes in this guide — <code>az login</code> honors the tenant's policies. Expect tokens to expire more often; re-run <code>az login</code> when a command replies “interaction required”

14 · Troubleshooting

SYMPTOM	CAUSE → FIX
<code>answered by: stub</code>	Endpoint/key/deployment name wrong in <code>.env</code> , or no network. The stub answering is the designed fallback — fix <code>.env</code> and rerun <code>testmodel</code>
<code>InsufficientQuota</code> on deployment create	Subscription lacks quota for that model — <code>az cognitiveservices usage list -l \$LOC -o table</code> shows what's available; substitute and update <code>.env</code>
<code>ServiceModelDeprecating</code>	That model version no longer accepts new deployments — deploy the current version instead
401 from the deployed console for yourself	You're signed in to a different tenant in that browser, or the registration's redirect URL doesn't match the app name — recheck §12.4
<code>AuthorizationFailed</code> on uploads	The role assignment hasn't propagated (wait a minute) or was scoped to the wrong storage account
Tests fail offline	Should never happen from a clean clone — check Python ≥ 3.12 and that Tesseract is installed (two OCR tests need it)
Everything is slow; reads hang	The repo sits in a cloud-synced folder — move it to a plain local path (§2 warning)
Corporate proxy	<code>export HTTPS_PROXY=http://proxy:port</code> — both pip and the platform honor it

15 · Appendix — the whole thing as one command sequence

```
# prerequisites installed ($2), then:
git clone https://github.com/git-bondal08/IQR_Agentic_AI.git IQR && cd IQR
python3 -m venv .venv && .venv/bin/pip install -e .
.venv/bin/python tests/fixtures/build_fixtures.py && .venv/bin/python -m pytest tests/ -q # 61 passed

az login && az account set --subscription "<sub>"
export LOC=eastus2 RG=rg-iqr-sox SA=stiqrsox<sfx> FOUNDRY=iqr-foundry-<sfx> SEARCH=iqr-search-<sfx>
bash scripts/azure/provision.sh # RG + storage + tables + Foundry + Search
# add deployments: gpt-5-mini, model-router, text-embedding-3-small ($7)
cp .env.example .env # fill from the three key commands ($8)
.venv/bin/python -m iqr.cli testmodel # answered by: foundry
# grant blob role + upload-batch the six data sets ($9)
.venv/bin/python scripts/azure/seed_knowledge.py # 12/12 indexed
.venv/bin/python -m iqr.cli eval --batch 3 # ALL GATES PASS
# App Service + settings + bundle + deploy + Easy Auth ($12)
```

Companions in the repository: **docs/UAT.md** (13 acceptance cases) · **docs/PRODUCTION_STANDARDS.md** (enterprise hardening ladder: managed identities, private endpoints, WORM retention, CI/CD gates) · **docs/design/IQR_Design_Document_v4.docx** (the architecture) · **docs/AZURE_FOUNDRY.md** (integration reference). The platform's own rule applies to this guide too: if a step's expected output doesn't match, stop and fix before continuing — every later step assumes the earlier ones are green.